

Corpus checker (PC-ACE data)

The PC-ACE data-reliability pipeline — an overview of all eight checks

Contents

1. What this GUI is	1
2. How your data must be organized	1
3. The eight checks	1
4. A suggested order of use	2
5. Which NLP parser is used	3
6. Reference.....	3

1. What this GUI is

The **Corpus checker** brings together eight computer-automated checks on the *reliability* of a collection of text documents collected via **PC-ACE** (Program for Computer-Assisted Coding of Events). Rather than analyzing content, these tools ask a different question: *is my corpus correctly built?* Are the filenames well formed? Is every document filed under the right event? Are the same people and places spelled consistently? Did my hand-written summaries leave out actors who appear in the sources? Are some documents simply copies of others?

The checks can be applied to an ongoing project (to catch problems while data collection is under way) or to a completed project (to audit what was collected). They were designed, in the words of the paper that introduced them, with "a fourth grader" in mind: a user with no computer science background.

2. How your data must be organized

This is the one prerequisite. Your documents must be grouped into **subfolders, one per cluster**. In the lynching project a "cluster" is an *event* and the documents are newspaper articles — but the tools are general: clusters may equally be topics (short stories), informants (field notes) or recipients (letters).

- One subfolder per cluster, holding only that cluster's documents. Folders may be named by event ID (3), by ID and name (3_Jim Cobb), or by ID, name and year (3_Jim Cobb_1918) — whatever convention you adopt, keep it consistent.
- Document filenames embed metadata separated by an underscore: newspaper, date, page, column — e.g. The Atlanta Constitution_01-22-1892_4_2.
- Event information may be appended after a *double* underscore __ — e.g. The Atlanta Constitution_01-22-1892_4_2__3 or The Atlanta Constitution_05-24-1918_2_1__Jim Cobb.
- The date embedded in the filename is what makes the time-based options work (and the "Filename embeds date" checkbox).

For **Find the missing character** you additionally need a second folder of *compilations* (the human-written summary of each cluster), one file per cluster, named after the cluster ID — e.g. 3.txt.

3. The eight checks

Each check has its own detailed TIPS; use the dropdown menu at the bottom of the GUI. This table is the map.

Check	What it flags	How
Check the filenames well-formedness	filenames that do not follow your naming convention, so the embedded metadata cannot be read reliably	filename parsing
Find the character	the characters (social actors: individuals, groups, organizations; also animals) present in your documents	WordNet
Find the character's ancestor	aggregates words into higher-level categories ("talk", "say", "write" are all verbs of communication)	WordNet, VerbNet or FrameNet
Find the missing character	actors who appear in a cluster's documents but are MISSING from its hand-written compilation	POS + NER
Check the character's name tag	names/locations spelled inconsistently within a cluster (Fleming vs Flemming, Coolidge vs Cooledge)	Levenshtein edit distance + NER
Find the intruder	documents filed in the WRONG cluster	POS + NER
Find the character's home	the most likely cluster for loose/unfiled documents	POS + NER, or date matching
Find the plagiarist	documents that are near-duplicates of each other (how much sources copied one another)	TF-IDF + cosine similarity

A note on knowledge bases. *Find the character* and *Find the character's ancestor* both open the Semantic aggregation GUI, where you choose the knowledge base: **WordNet** (synsets), **VerbNet** (verb classes) or **FrameNet** (semantic frames). Which one to use depends on what you are aggregating: WordNet organizes *nouns* into an is-a hierarchy and is what produces the list of social actors, while VerbNet and FrameNet come into their own for *verbs* and for the frames (and frame roles) in which actors appear — for example the "talk", "say", "write" verbs of communication above.

One caveat for this pipeline: *Find the missing character* and *Find the intruder* compare your documents against the social-actor list stored as social-actor-list.csv in the lib/wordLists folder, which is built from WordNet. So whichever knowledge base you explore for aggregation, it is the WordNet-derived actor list that those two checks actually read.

4. A suggested order of use

The checks are independent — run whichever you need — but they build on each other naturally:

- Start with **filenames well-formedness**: everything downstream reads metadata out of the filenames.
- Build your vocabulary with **Find the character** and **Find the character's ancestor**: these produce the actor lists the later checks rely on.

- Audit cluster integrity with **Find the intruder** (wrong folder), **Find the character's home** (unfiled documents) and **Check the character's name tag** (spelling variants).
- Audit your summaries with **Find the missing character**.
- Finally, measure redundancy with **Find the plagiarist**.

5. Which NLP parser is used

The checks that parse text (*Find the missing character* and *Find the intruder*) use the NLP package you selected in the NLP Suite setup — **Stanford CoreNLP, Stanza, or spaCy**. You are only asked for the Stanford CoreNLP directory when CoreNLP is the package you selected. Note that named-entity results differ somewhat between packages, since each uses a different model: if you switch package mid-project, re-check a few clusters before comparing old and new output.

6. Reference

These tools, and the lynching data used to develop them (Georgia, 1875–1935; some 1,000 newspaper articles), are described in full in:

*Franzosi, R., Dong, W., & Dong, Y. (2021). "Qualitative and quantitative research in the humanities and social sciences: how natural language processing (NLP) can help." **Quality & Quantity**. <https://doi.org/10.1007/s11135-021-01235-2>*

See also: the individual TIPS for each check, and the PC-ACE tools (PC-ACE Tables Analyzer, PC-ACE Data Validation & Cleaning).